

The lakehouse and the platform

Control plane and compute plane, Delta Lake and the transaction log, and the Unity Catalog object model.

A lakehouse is not a data lake with better file formats. It is a data lake that has been given **a transaction log**.

Why the two older architectures both fell short

DATA WAREHOUSE

Reliable but closed

- Transactions, schema enforcement, fast SQL
- Proprietary storage format
- Expensive at scale
- Poor fit for unstructured data and ML

DATA LAKE

Open but unreliable

- Cheap object storage, any file type
- No transactions, so partial writes happen
- No schema enforcement, so quality drifts
- Reader sees half a job's output

THE PROBLEM THE LAKEHOUSE SOLVES

Most organisations ran both and copied data between them. Two systems, two copies, and a nightly window where the two disagreed.

What the lakehouse adds

- ◆ **Open storage.** Parquet files in your own cloud object storage. Any engine can read them.
- ◆ **ACID transactions.** A write either commits or it does not. Readers never see a half-finished job.
- ◆ **Schema enforcement and evolution.** Bad data is rejected at write time rather than discovered later.
- ◆ **One copy, both workloads.** BI and machine learning read the same tables. No nightly sync.

ON THE EXAM

Every one of these is delivered by Delta Lake, not by the platform around it. That distinction gets tested.

Two planes, and what lives in each

CONTROL PLANE	COMPUTE PLANE
The web application and workspace UI	Clusters and SQL warehouses
Notebook source and query definitions	Your actual table data
Job and pipeline configuration	Driver and executor processes
Cluster management and the REST API	The work of running your query
Managed by Databricks	Where processing happens

ON THE EXAM

The control plane holds the backend services Databricks manages. The compute plane is where your data is processed.

Classic compute plane or serverless compute plane?

CLASSIC COMPUTE PLANE

In your cloud account

- Resources run in your own AWS, Azure or GCP account
- In your own network, which you can configure
- Slower to start; you wait for VMs
- Choose when you need network control or compliance isolation

SERVERLESS COMPUTE PLANE

In the Databricks account

- Resources run in a Databricks-managed compute layer
- Always in the same cloud region as your workspace
- Starts in seconds; no VMs to provision
- Choose for speed and for not managing infrastructure

THE TIEBREAKER

Both are compute planes. Serverless does not move work into the control plane; it moves it into a different compute plane.

Two consequences worth remembering

- ◆ **Classic compute has no open inbound ports and no public IP addresses.** Clusters reach the control plane through an outbound HTTPS tunnel.
- ◆ **This is called secure cluster connectivity,** and it is the default on all new workspaces.
- ◆ **Serverless runs in the same region as your workspace,** so processing does not cross a regional boundary.
- ◆ **Notebook code lives in the control plane.** That surprises people who assume everything sits in their own account.

ON THE EXAM

Data residency questions turn on the compute plane. Code and configuration questions turn on the control plane.

A Delta table is ordinary Parquet files, plus a folder called **_delta_log** that says which of those files currently count.

What happens when you write

01

Write files

New Parquet files are written to storage. Nothing yet reads them.

02

Commit

A numbered JSON file is added to `_delta_log` listing what changed.

03

Visible

That commit is the moment the write becomes visible to readers.

04

Checkpoint

Every 10 commits by default, a Parquet snapshot of the log is written.

WHY ATOMICITY WORKS

Files exist before the commit but do not count. That is why a failed job leaves no partial data visible.

What the log gives you

- ◆ **Atomic writes.** Readers see the state before your job or the state after it, never during.
- ◆ **Time travel.** Every version is still described in the log, so you can query the table as it was.
- ◆ **Audit history.** `DESCRIBE HISTORY` shows every operation, who ran it and when.
- ◆ **Data skipping.** The log stores per-file min and max statistics, so irrelevant files are never opened.

ON THE EXAM

Time travel is bounded by file retention. `VACUUM` deletes old files and travel past that point stops working.

Delta table or plain Parquet files?

PLAIN PARQUET

A pile of files

- No transactions; partial writes are visible
- No history and no rollback
- Row-level updates mean rewriting whole files by hand
- Schema is whatever each file happens to contain

DELTA TABLE

Files plus a log

- ACID commits, so writes are all or nothing
- Time travel, DESCRIBE HISTORY and RESTORE
- MERGE, UPDATE and DELETE on rows
- Schema enforced on write, and evolvable on purpose

THE TIEBREAKER

Delta is the default table format on Databricks. If a question describes reliability, history or row-level changes, the answer involves Delta.

The Unity Catalog namespace

01

Metastore

Top-level container.
One per cloud region.

02

Catalog

First level of the name.
Often an environment
or a business unit.

03

Schema

Second level. A project,
a use case, or a team's
area.

04

Table or volume

Third level. Also views,
functions and models.

SAY IT OUT LOUD

Three-level namespace: `catalog.schema.table`. The metastore is above it and is never part of the name you write.

What sits where

LEVEL	WHAT IT CONTAINS
Metastore	Catalogs, plus storage credentials, external locations, connections, shares, recipients and providers
Catalog	Schemas, and nothing else directly
Schema	Tables, views, volumes, functions and models
Table or volume	Managed by Unity Catalog, or external to it

ON THE EXAM

Storage credentials and external locations hang off the metastore, not off a catalog. That placement is examinable.

Managed or external

Introduced here; the full treatment is module 27

- ◆ **Managed.** Unity Catalog controls both governance and the underlying files, including their lifecycle.
- ◆ **Drop a managed table and the data files are deleted too.**
- ◆ **External.** Unity Catalog governs access; you keep control of the files and where they live.
- ◆ **Drop an external table and only the registration goes.** The files remain untouched.

ON THE EXAM

Both tables and volumes come in managed and external forms. Databricks recommends managed unless you have a reason not to.

DBFS is deprecated

Both the root and mounts

- ◆ **DBFS root and DBFS mounts are deprecated.** New accounts are provisioned without access to either.
- ◆ **The DBFS root is readable by every user in the workspace.** There is no way to restrict it.
- ◆ **Mounts bypass Unity Catalog entirely.** A different access model with different credentials.
- ◆ **Use instead:** Unity Catalog volumes, external locations, or workspace files.

ON THE EXAM

The name survives in one place. `dbfs:/` is still an optional scheme for addressing a Unity Catalog volume.

hive_metastore or Unity Catalog?

HIVE_METASTORE

The legacy catalog

- Scoped to one workspace only
- Managed tables land on the DBFS root
- Legacy table access control lists
- Appears as a catalog for backward compatibility

UNITY CATALOG

The current model

- Shared across workspaces in a region
- Managed tables use secure managed storage
- GRANT, REVOKE, DENY and ABAC policies
- Lineage, audit and search across everything

THE TIEBREAKER

If a question mentions sharing data between workspaces, or governing it centrally, the legacy metastore cannot do it.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Where does the web application run	Control plane
Where is my data processed	Compute plane, classic or serverless
Partial writes, rollback, or history	Delta Lake and the transaction log
Query the table as it was last Tuesday	Time travel, if VACUUM has not run
Share governed data across workspaces	Unity Catalog, never hive_metastore
Files on a path, governed	A Unity Catalog volume, never a DBFS mount

ON THE EXAM

Six sentences. If you can map each to its answer without hesitating, this module has done its job.

A bank's security review asks the data platform team to confirm that no customer data leaves the bank's own cloud account. The team uses classic compute for all production jobs, and stores every table in buckets the bank owns.

The reviewer then asks a second question. She wants to know where the source code of the notebooks and the definitions of the scheduled jobs are stored.

How should the team answer the second question?

- ☐ A Notebook source and job definitions are stored in the workspace storage bucket in the bank's own cloud account
- ☐ B Notebook source and job definitions are stored in the Databricks control plane
- ☐ C Notebook source is stored on the cluster driver node and is discarded when the cluster terminates
- ☐ D Notebook source and job definitions are stored in the serverless compute plane

◆ PAUSE AND DECIDE

How should the team answer the second question?

- A

In the workspace storage bucket in the bank's account

Tempting, because the first answer genuinely was 'in our account'. But this is where the two questions diverge. Data sits in the bank's storage; code does not.
- C

On the driver node, discarded on termination

Confuses execution with storage. Code is sent to the driver to run, but it is not stored there, or nothing would survive a restart.
- B

In the Databricks control plane

Correct. Notebook source, query definitions and job configuration are all backend services that Databricks manages in the control plane.
- D

In the serverless compute plane

The team uses classic compute, so no serverless compute plane is even involved. Compute planes process data; they do not store definitions.

THE SIGNAL

The reviewer asked two questions that sound like one. Data residency and code residency have different answers.

An engineer joins a company and finds that a nightly job writes a customer file containing names and account numbers to a path on the DBFS root. A second job reads it back the next morning.

It has worked reliably for two years. The engineer raises it with her manager, who asks what is actually wrong with it, given that it has never failed.

What is the strongest reason to change this?

- A The DBFS root is slower than cloud object storage, so the job takes longer than it needs to
- B Every user in the workspace can read anything on the DBFS root, and this file contains account numbers
- C The DBFS root has a size limit that this file will eventually exceed
- D Files on the DBFS root are deleted automatically after ninety days

◆ PAUSE AND DECIDE

What is the strongest reason to change this?

A It is slower than cloud object storage
The DBFS root is backed by cloud object storage, so there is no meaningful performance gap. And the manager's question is about risk, not speed.

C It will exceed a size limit
Invented. There is no such limit, and a fabricated constraint is a common distractor shape.

B Every workspace user can read it, and it holds account numbers
Correct. The DBFS root is readable by all workspace users with no way to restrict it. Two years without failure says nothing about exposure.

D Files are deleted after ninety days
Also invented. Nothing on the DBFS root expires on a schedule.

THE SIGNAL

'It has never failed' is about reliability. The risk here is disclosure, which produces no failures at all until it does.

A nightly job overwrote a 400 million row Delta table with the output of a transformation that had a bug in it. The table now contains roughly 12 million rows, all of them wrong. The error was noticed nine hours later.

The table is written to daily and has never had VACUUM run against it. The team needs the previous day's data back and wants the fastest correct route.

What should the team do?

- ☐ A Re-run the upstream pipeline from raw source data to rebuild the table
- ☐ B Restore the storage bucket from the cloud provider's most recent snapshot
- ☐ C Use RESTORE to return the table to the version before the bad write
- ☐ D Recover the deleted Parquet files from the storage bucket and re-register them

◆ PAUSE AND DECIDE

What should the team do?

- A

 Re-run the upstream pipeline from raw
It would work, and that is what makes it tempting. But it reprocesses 400 million rows to undo a change the log can reverse by pointing at an earlier version.
- B

 Restore the bucket from a cloud snapshot
Blunt and risky. It rolls back everything in that bucket, including other tables, to whenever the snapshot was taken.
- C

 RESTORE to the version before the bad write
Correct. The old files still exist because VACUUM has never run, so this is a metadata operation. No data is reprocessed or moved.
- D

 Recover deleted Parquet files and re-register them
Nothing was deleted. An overwrite writes new files and marks the old ones as removed in the log; they remain on disk until VACUUM.

THE SIGNAL

'VACUUM has never run' is the sentence that decides this. It guarantees the previous version's files are still there.

A team receives a daily delivery of PDF contracts into cloud object storage. A Python library that does not use Spark reads them by file path and extracts text.

The company has adopted Unity Catalog and requires that access to every data asset is granted centrally and appears in audit logs. The PDFs must stay in the existing bucket, because a separate legal system also writes to it.

How should the PDFs be made available?

- ☐ A Mount the bucket to DBFS so the library can read the files by path
- ☐ B Create an external location for the bucket, then an external volume over it
- ☐ C Create a managed volume and copy the PDFs into it each day
- ☐ D Register the PDFs as an external Delta table so they can be governed

◆ PAUSE AND DECIDE

How should the PDFs be made available?

- A

 Mount the bucket to DBFS
It would give the library path access, which is why it tempts. But mounts are deprecated and bypass Unity Catalog, so the central grant and audit requirement fails.
- B

 External location, then an external volume
Correct. Volumes give governed path-based access for non-tabular files, and external keeps the PDFs in the existing bucket the legal system also writes to.
- C

 Managed volume, copying files in daily
Governed and path-addressable, but it duplicates every PDF and adds a job that can fail. The requirement was to leave them where they are.
- D

 External Delta table
A Delta table describes rows and columns. PDFs are not tabular, and a table gives no path for a non-Spark library to open.

THE SIGNAL

Two constraints, each eliminating a different option. 'Reads them by file path' rules out a table. 'Must stay in the existing bucket' rules out managed.

Delta Lake gives you **reliability**. Unity Catalog gives you **governance**. The planes tell you **where things run**. Every later module sits on one of those three.